CS-GY 6033

Design and Analysis of Algorithms I

Lecture 6 | Part 1

Theory of Locality Sensitive Hashing

Why does LSH work?

- Two approaches to understanding LSH.
- **1) Hashing view.**
- 2) Dimensionality reduction view.

Standard Hashing

- A **hash function** $h : \mathcal{X} \to \mathbb{Z}$ takes in an object from $\mathcal{X}$ and returns a bucket number.

Standard Hashing

- **Collision**: two different objects have same hash.
- Usually, collisions are **bad**.
- Want similar things to have very different hashes.

Locality Sensitive Hashing

- But in NN search, we want “close” items to be in the **same bucket** (have same hash).
- “Far” items should be in **different buckets** (have different hash).

Locality Sensitive Hashing

- Let r be a distance we consider “close”.
- Let cr (with $c > 1$) be a distance we consider “far”.
- Suppose H is a **family** of hash functions.

LSH Family

- H is an **LSH family** if when h is randomly drawn from H:

$$\mathbb{P}(h(x) = h(y)) \geq p_1 \quad \text{when } d(x, y) \leq r$$
$$\mathbb{P}(h(x) = h(y)) \leq p_2 \quad \text{when } d(x, y) \geq cr$$

where $p_1 > p_2$.

Main Idea

If x and y are close, the probability that they hash to the **same** bin is not too small. If they are far, the probability is not too large.

Example: Random Projections

- We have seen one LSH family: random projections followed by binning.
- H has infinitely-many hash functions, one for each direction $\vec{u}$:

$$h_{\vec{u}}(\vec{p}) = \left\lfloor \frac{\vec{u} \cdot \vec{p}}{w} \right\rfloor,$$

$\vec{u} \cdot \vec{p} = \|\vec{u}\| \cdot \|\vec{p}\| \cdot \cos\theta$

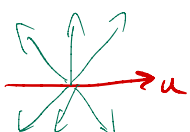
width

Example: Random Projections

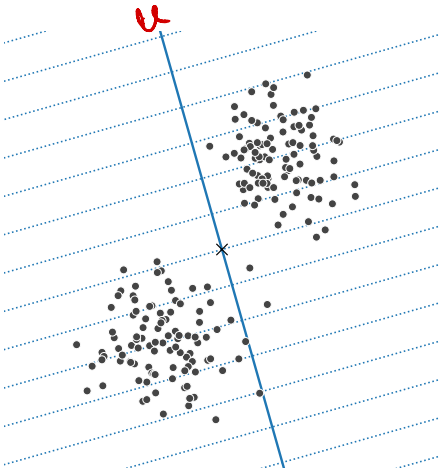
- Suppose a random hash function h is chosen.
- Claim:

$$\mathbb{P}(h(x) = h(y)) \geq \frac{1}{2} \quad \text{when } d(x, y) \leq w/2$$

$$\mathbb{P}(h(x) = h(y)) \leq \frac{1}{3} \quad \text{when } d(x, y) \geq 2w$$

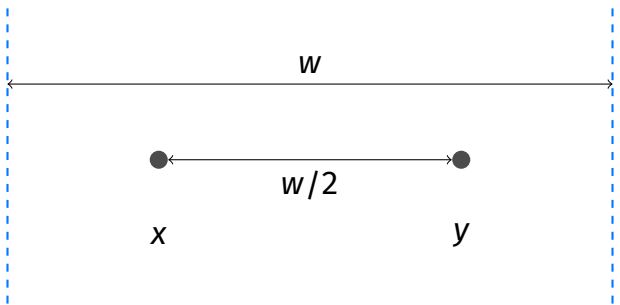


Intuition



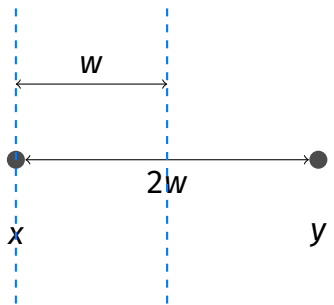
Proof: Close

- In worst case, grid is orthogonal to line between points.



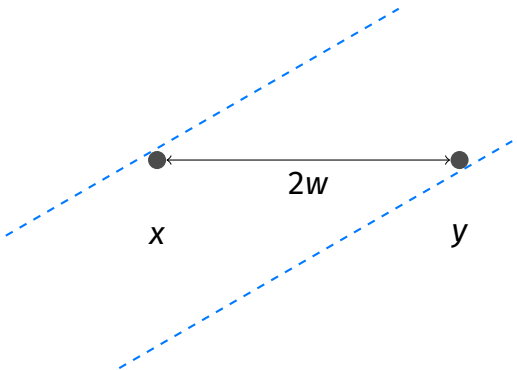
Proof: Far

- Only possible if grid is close to parallel.



Proof: Far

- Angle must be below 30˚.



Amplification

- Lots of points have same hash.
- To be more selective, randomly select k hash functions for cell id.

$$\text{cell-id}(x) = (h_1(x), h_2(x), \ldots, h_k(x))$$

Example: Random Projections

- In case of random projections.

$$\text{cell-id}(\vec{p}) = \left(\underbrace{\left\lfloor \frac{\vec{u}^{(1)} \cdot \vec{p}}{w} \right\rfloor}_{h_1}, \underbrace{\left\lfloor \frac{\vec{u}^{(2)} \cdot \vec{p}}{w} \right\rfloor}_{h_2}, \ldots, \underbrace{\left\lfloor \frac{\vec{u}^{(k)} \cdot \vec{p}}{w} \right\rfloor}_{h_k} \right)$$

$\text{cell-id}(\vec{p}) \in \mathbb{R}^k$

$\vec{p} \in \mathbb{R}^d$

Collision Probability

- Remember:

$$P(h(x) = h(y)) \geq p_1 \text{ if close.}$$
$$P(h(x) = h(y)) \leq p_2 \text{ if far.}$$

- Collision occurs if $h_i(x) = h_i(y)\ \forall i \in \{1, \ldots, k\}$.

for x and y all the $h_i(x) = h_i(y)$ should hold.

- Probability of collision...
  - if close: $\geq p_1^k$
  - if far: $\leq p_2^k$

what k should we set?

Choosing k

- Want prob. of far points colliding to be small.
- Say, $1/n$.
- Set $p_2^k = 1/n$. Then

$$k = \log_{p_2} \frac{1}{n} = -\frac{\log n}{\log p_2}$$

Main Idea

We can use $k = \Theta(\log n)$ hash functions.

Main Idea

When using random projections as hash functions, we can use $k = \Theta(\log n)$ directions. This is usually much less than d.

But wait...

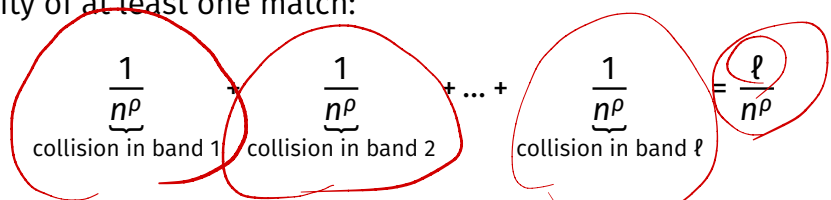
- Probability of close points colliding is p_1^k.
- Let $p_1 = p_2^\rho$. We'll have $\rho < 1$, since $p_2 < p_1$.
- Since $p_2^k = \frac{1}{n}$, we have $p_1^k = \frac{1}{n^\rho}$.
- This is **very small**.

Banding

- Before: one set of k hash functions.
- With **banding**: keep ℓ sets (**bands**) of k hash functions.
- To query NN of p, find points that are in the same cell as p in *any* of the bands.

Banding

- Probability of at least one match:



- Want this to be ≈ 1, so:

$$\ell = n^\rho$$

Main Idea

We should set the number of bands to be n^{ρ}. ρ depends on c, and is usually not small. For random projections, $\rho \approx .63$.

Analysis

- How efficient is LSH?
- Worst case, everything hashes to same bin: $O(n)$.
- In practice, much better.
- Requires **a lot** of memory. $\Theta(\ell n)$.

$\ell = n^{\rho}$

Other Distances

- LSH works for many different similarity measures.
- Random projections are for Euclidean distances.
- But other hashing approaches work for cosine distance, Jaccard distance, etc.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 2

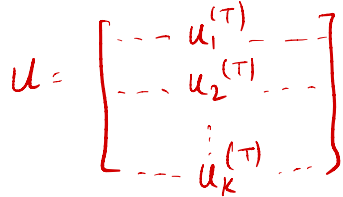
The Johnson-Lindenstrauss Lemma

Why does LSH work?

- Two approaches to understanding LSH.
- 1) Hashing view.
- **2) Dimensionality reduction view.**

Main Idea

The **Johnson-Lindenstrauss Lemma** says that, given n points in $\mathbb{R}^d$, you can reduce the dimensionality to $k \approx \log n$ while still preserving relative distances by randomly projecting onto a set of k unit vectors.



Claim

The **Johnson-Lindenstrauss Lemma** (Informal). Let X be a set of n points in $\mathbb{R}^d$. Let U be a matrix whose $k = O(\log(n)/\epsilon^2)$ rows are Gaussian random vectors in $\mathbb{R}^d$. Then for every $\vec{x}, \vec{y} \in X$,

$$\|\vec{x} - \vec{y}\| \leq (1 \pm \epsilon)\|U\vec{x} - U\vec{y}\|$$

LSH and J-L

- In LSH, we use $k = O(\log n)$ hash functions.
- If these hash functions are random projections, the J-L lemma tells that distances are largely preserved.

A Different View of LSH

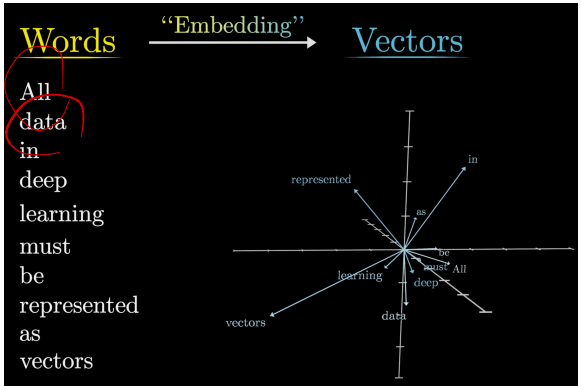
- Given $p \in \mathbb{R}^d$, randomly project to $\mathbb{R}^k$ with $k \approx \log n$.
- Let new coordinates be $(y_1, y_2, \ldots, y_k)$.
- Use standard grid to assign cell id.

Main Idea

LSH (for Euclidean distances) (without banding) can be viewed as dimensionality reduction by random projections, followed by binning into a standard grid.

Aside: J-L and LLMs

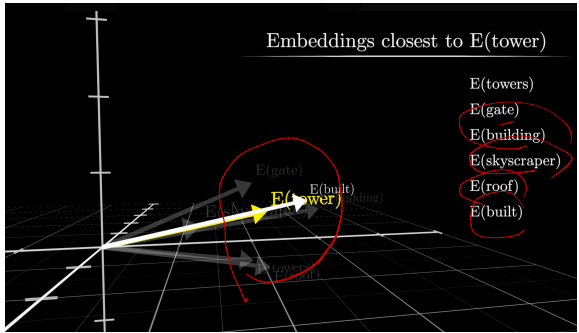
- Words are embedded into a d-dimensional space[1].



[1]Figures form 3Blue1Brown Youtube channel

Aside: J-L and LLMs

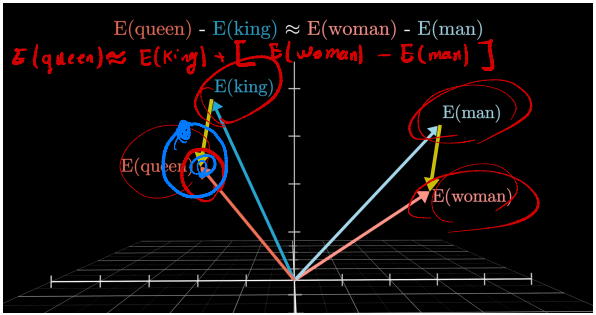
- Words that are similar are embedded close to each other; a larger inner product.



- Words that are very different (carry different concept) are embedded far from each other; small inner product.

Aside: J-L and LLMs

- Semantic embeddings allows us to do math



- Higher dimensional space $\implies$ encoding of more independent concepts.

Aside: J-L and LLMs

- Consequence of JL Lemma: the number of vectors that we can cramp into a space that are **nearly** perpendicular **grows exponentially** with the number of dimensions.

- Very significant for LLMs: LLMs benefit from associating independent ideas to nearly perpendicular directions. Store many more ideas than there are dimensions that is allowed.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 3

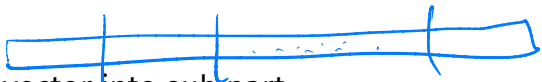
NN in Practice

In Practice

- LSH is an important idea.
- Good performance in practice.
- But heuristic approaches are often faster.
- `faiss` and `annoy`, among others.

Other Approaches

- Navigable small worlds.
  - Build a sparse graph in embedding space.
  - Similarity search in embedding space using the graph.
- Product quantization.
  - Break each embedding vector into sub part.
  - For each subspace, run k-means to find centers, use them as IDs.
  - Do the search using this compressed representations.
- Hierarchical k-means.



CS-GY 6033
Design and Analysis of Algorithms I

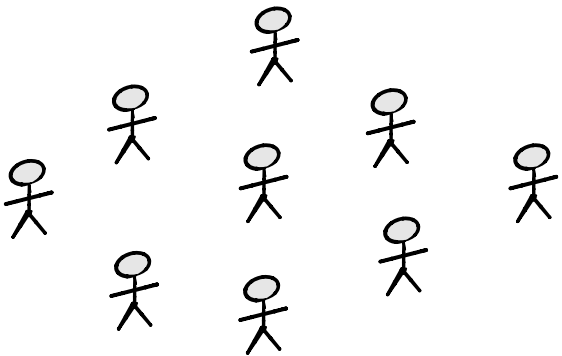
Lecture 6 | Part 4

Graphs

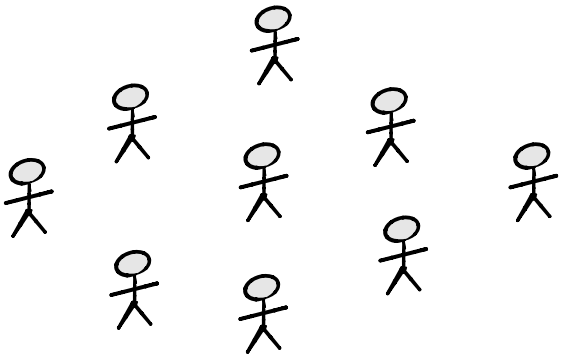
Data Types

- **Feature vectors**
  - We care about attributes of individuals.
- **Graphs**
  - We care about relationships between individuals.
- **Graph Neural Networks**
  - Capture both features and relationships.

Example: Facebook



Example: Twitter

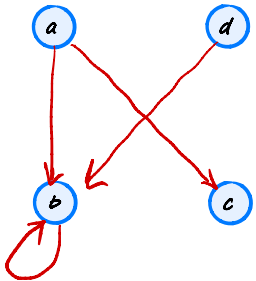


Definition

A **directed graph** (or **digraph**) G is a pair (V, E) where V is a finite set of **nodes** (or **vertices**) and E is a set of ordered pairs (the **edges**).

Example:

$V = \{a, b, c, d\}$
$E = \{(a, c), (a, b), (d, b), (b, d), (b, b)\}$



Directed Graphs (More Formally)

E is a subset of the **Cartesian product**, $V \times V$.

Example:
$\{a, b, c\} \times \{1, 2\} = \{(a,1), (a,2), (b,1), (b,2), (c,1), (c,2)\}$

Consequences

Because the edge set of a directed graph is allowed to be *any* subset of $V \times V$:

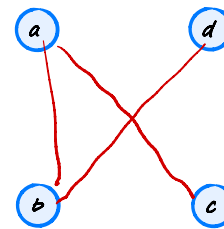
- the edges have directions.
  - e.g., (a, b) is "from a to b"
- can have "opposite" edges.
  - e.g., (a, b) and (b, a).
- can have "self-loops"
  - e.g., (a, a)

Definition

An **undirected graph** G is a pair (V, E) where V is a finite set of **nodes** (or **vertices**) and E is a set of unordered, distinct pairs (the **edges**).

Example:

$V = \{a, b, c, d\}$
$E = \{\{a, c\}, \{a, b\}, \{d, b\}\}$



Undirected Graphs (More Formally)

An edge in an undirected graph is a set $\{u, v\}$ where $u \neq v$. This has consequences:

- the edges have **no direction**.
  - e.g., $\{a, b\}$ is **not** "from" a "to" b.
- **cannot** have "opposite" edges.
  - e.g., $\{a, b\}$ and $\{b, a\}$ are the same.
- **cannot** have "self-loops"
  - e.g., $\{a, a\}$ is not a valid edge

Notational Note

Although edges in undirected graphs are sets, we typically write them as pairs: (u, v) instead of $\{u, v\}$.

Summary

- Edges have direction:
  - Directed: **yes**
  - Undirected: **no**
- Self-loops, (u, u)?
  - Directed: **yes**
  - Undirected: **no**
- Opposite edges, (u, v) and (v, u)?
  - Directed: **yes**
  - Undirected: **no** (they are the same edge)

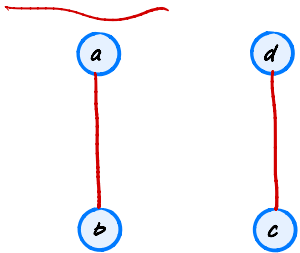
Note

Neither directed nor undirected graphs can have **duplicate edges**[2]

[2]There are other definitions which allow duplicate edges.

Note

Graphs don’t need to be “connected”



Exercise

What is the greatest number edges possible in a **directed** graph?

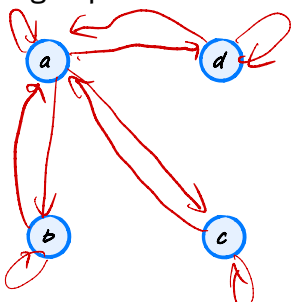
n nodes.

$= n^2$

undirected: $\frac{n(n-1)}{2}$

Counting Edges

What is the greatest number edges possible in a **directed** graph?



$|V \times V| = n^2$

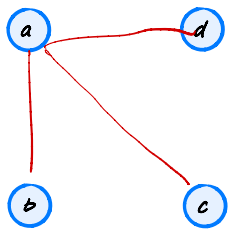
Exercise

What is the greatest number edges possible in an **undirected** graph?

$$\frac{n(n-1)}{2}$$

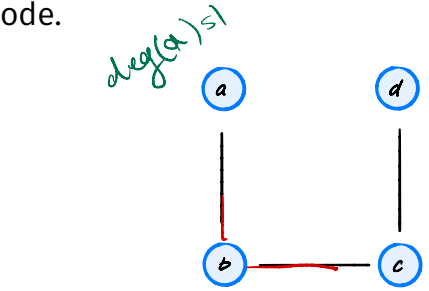
Counting Edges

What is the greatest number edges possible in an **undirected** graph?



Degree

The **degree** of a node in an undirected graph is the number of edges containing that node.



$E = \{(a,b), (b,c), (c,d)\}$

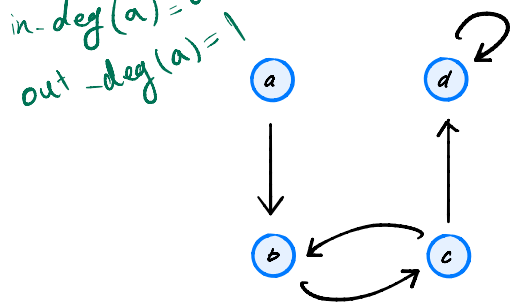
In-Degree/Out-Degree

The **in-degree** of a node in an directed graph is the number of edges **entering** that node.

The **out-degree** of a node in an directed graph is the number of edges **leaving** that node.

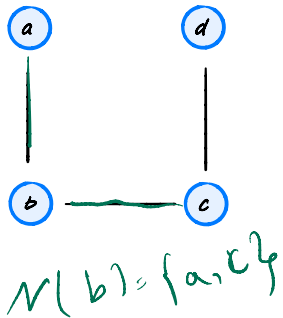
The **degree** of a node in a directed graph is the in-degree + out-degree.

Examples



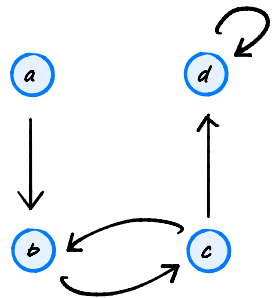
Neighbors

Definition: in an undirected graph, the set of **neighbors** of a node u is the set of all nodes which share an edge with u.



Predecessors

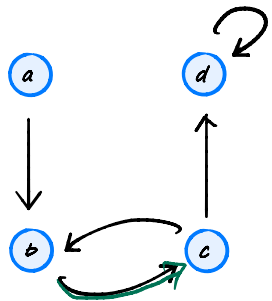
Definition: in an directed graph, the set of **predecessors** of a node *u* is the set of all nodes which are at the **start** of an edge **entering** *u*.



$pred(b) = \{a, c\}$

Successors

Definition: in an directed graph, the set of **successors** of a node *u* is the set of all nodes which are at the **end** of an edge **leaving** *u*.

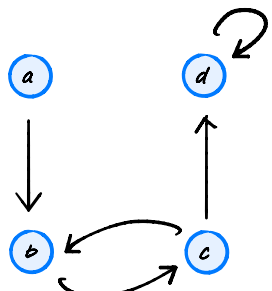


succ(b) = {c}

succ(d) = {d}

A Convention

In a directed graph, the **neighbors** of u are the **successors** of u.



CS-GY 6033
Design and Analysis of Algorithms I

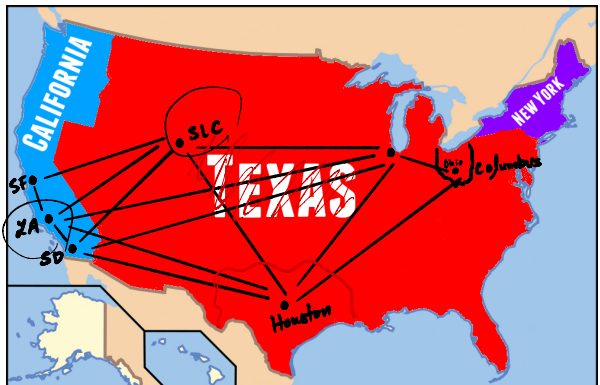
Lecture 6 | Part 5

Paths

Example

- Consider a graph of direct flights.
- Each node is an airport.
- Each edge is a direct flight.
- Should the graph be directed or undirected?

Example



Example

- Can we get from San Diego to Columbus?
- Not with a single edge.
- But with a **path**.

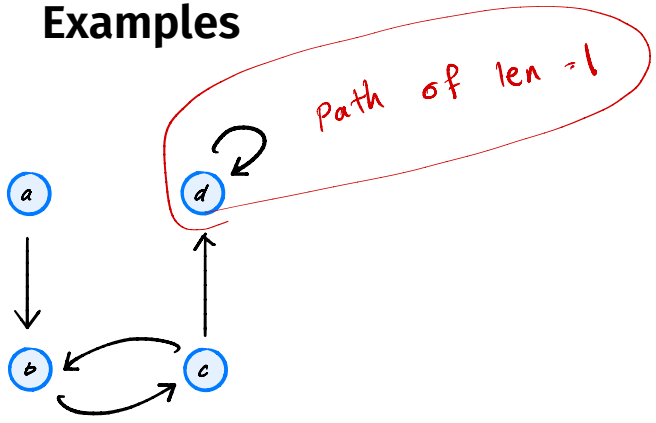
Definition

A **path** from u to u' in a (directed or undirected) graph $G = (V, E)$ is a sequence of one or more nodes $u = v_0, v_1, \ldots, v_k = u'$ such that there is an edge between each consecutive pair of nodes in the sequence.

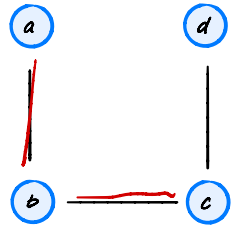
Path Length

Definition: The **length** of a path is the number of nodes in the sequence, minus one. Paths of length zero are possible!

Examples



Examples



$P_1 : a$ $\quad len(P_1) = 0$

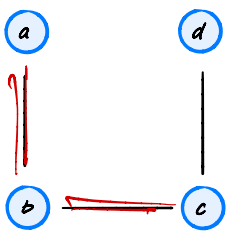
$P_2 = a\ b\ c$ $\quad len(P_2) = 2$

Note

Paths **can** go through the same node more than once!

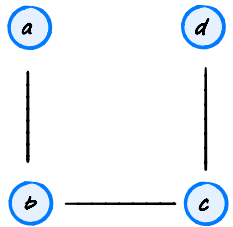
P_1: a b c b a

Len(P_1) = 4



Simple Paths

Definition: A **simple path** is a path in which every node is unique.

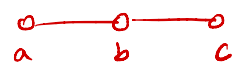


Reachability

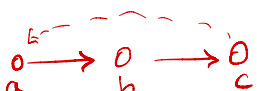
Definition: node *v* is **reachable** from node *u* if there is a path from *u* to *v*.

Reachability and Directedness

- If G is undirected, reachability is symmetric.
  - If u reachable from v, then v reachable from u.



- If G is directed, reachability is **not** symmetric.
  - If u reachable from v, then v may/may not be reachable from u.



Important Trivia

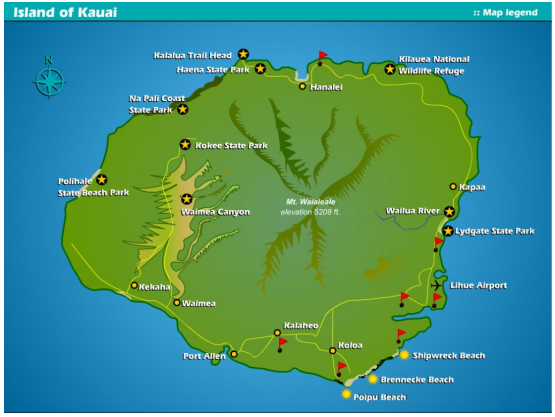
- In any graph, any node is **reachable** from **itself**.

CS-GY 6033
Design and Analysis of Algorithms I

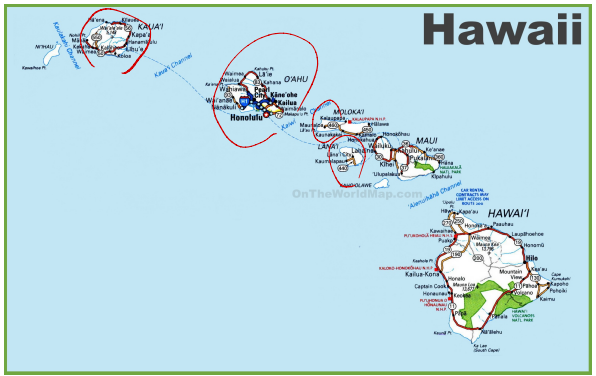
Lecture 6 | Part 6

Connected Components

Example


Island of Kauai
:: Map legend
Kalalua Trail Head
Haena State Park
Kilauea National
Wildlife Refuge
Hanalei
Na Pali Coast
State Park
Kokee State Park
Polihale
State Beach Park
Kapaa
Mt. Waialeale
elevation 5208 ft.
Waimea Canyon
Wailua River
Lydgate State Park
Kekaha
Lihue Airport
Waimea
Kalaheo
Koloa
Port Allen
Shipwreck Beach
Brennecke Beach
Poipu Beach

Example


Hawaii
KAUA'I
O'AHU
MOLOKA'I
MAUI
HAWAI'I
Honolulu
OnTheWorldMap.com

Connectedness

A graph is **connected** if every node *u* is reachable from every other node *v*. Otherwise, it is **disconnected**.

Equivalent: there is a path between every pair of nodes.

Connected Components

A **connected component** is a maximally-connected set of nodes.

I.e., if $G = (V, E)$ is an undirected graph, a connected component is a set $C \subset V$ such that

- any pair $u, u' \in C$ are reachable from one another; and
- if $u \in C$ and $z \notin C$ then u and z are not reachable from one another.

Exercise

What are the connected components?

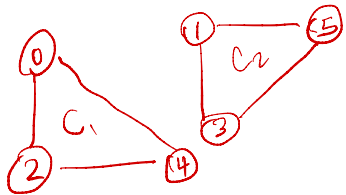
$$V = \{0, 1, 2, 3, 4, 5, 6\}$$
$$E = \{(0, 2), (1, 5), (3, 1), (2, 4), (0, 4), (5, 3)\}$$

Example

What are the connected components?

$V = \{0, 1, 2, 3, 4, 5, 6\}$

$E = \{(0, 2), (1, 5), (3, 1), (2, 4), (0, 4), (5, 3)\}$



CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 7

Adjacency Matrices

Representations

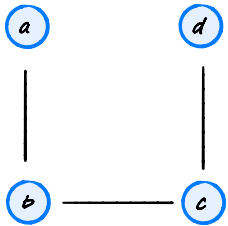
- How do we **store** a graph in a computer's memory?

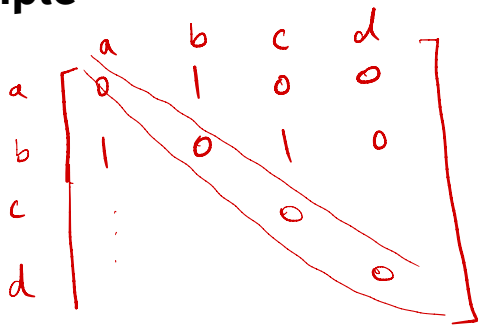
- Three approaches:
  1. Adjacency matrices.
  2. Adjacency lists.
  3. "Dictionary of sets"

Adjacency Matrices

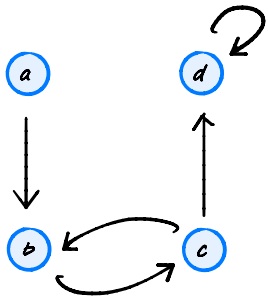
- Assume nodes are numbered 0, 1, ..., $|V| - 1$
- Allocate a $|V| \times |V|$ (Numpy) array
- Fill array as follows:
  - `arr[i,j] = 1` if $(i, j) \in E$
  - `arr[i,j] = 0` if $(i, j) \notin E$

Example





Example



	a	b	c	d
a	0	1	0	0
b	0	0	1	0
c	0	1	0	1
d	0	0	0	1

Observations

- If G is undirected, matrix is symmetric.
- If G is directed, matrix may not be symmetric.

Time Complexity

operation	code	time
edge query	`adj[i,j] == 1`	$\Theta(1)$
degree(i)	`np.sum(adj[i,:])`	$\Theta(\|V\|)$

Space Requirements

- Uses $|V|^2$ bits, even if there are very few edges.
- But most real-world graphs are **sparse**.
  - They contain many fewer edges than possible.

Example: Facebook

- Facebook has 2 billion users.

$$
\begin{aligned}
(2 \times 10^9)^2 &= 4 \times 10^{18} \text{ bits} \\
&= 500 \text{ petabits} \\
&\approx 6500 \text{ years of video at 1080p} \\
&\approx 60 \text{ copies of the internet as it was in 2000}
\end{aligned}
$$

Adjacency Matrices and Math

- Adjacency matrices are useful mathematically.
- Example: (i, j) entry of A^2 gives number of hops of length 2 between i and j.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 8

Adjacency Lists

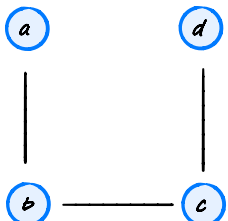
What's Wrong with Adjacency Matrices?

- Requires $\Theta(|V|^2)$ storage.
- Even if the graph has no edges.
- **Idea**: only store the edges that exist.

Adjacency Lists

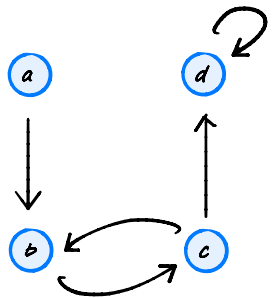
- Create a list `adj` containing $|V|$ lists.
- `adj[i]` is list containing the neighbors of node i.

Example




```
[ [b],
  [a,c],
  [b,d],
  [c]
]
```

Example




```
[ [b],
  [c],
  [b,d],
  [d]
]
```

Observations

- If G is undirected, each edge appears twice.
- If G is directed, each edge appears once.

Time Complexity

operation	code	time
edge query	`j in adj[i]`	$\Theta(\text{degree}(i))$
degree(i)	`len(adj[i])`	$\Theta(1)$

Space Requirements

- Need $\Theta(|V|)$ space for outer list.
- Plus $\Theta(|E|)$ space for inner lists.
- In total: $\Theta(|V| + |E|)$ space.

before : $\Theta(|V|^2)$

Example: Facebook

- Facebook has 2 billion users, 400 billion friendships.
- If each edge requires 32 bits:

$$
\begin{aligned}
&(2 \text{ bits} \times 200 \times (2 \text{ billion}) \\
&= 64 \times 400 \times 10^9 \text{ bits} \\
&= 3.2 \text{ terabytes} \\
&= 0.04 \text{ years of HD video}
\end{aligned}
$$

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 9

Dictionary of Sets

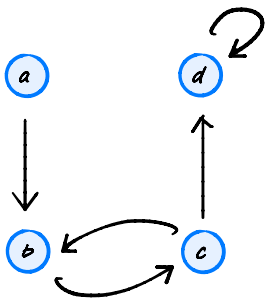
Tradeoffs

- Adjacency matrix: fast edge query, lots of space.
- Adjacency list: slower edge query, space efficient.
- Can we have the best of both?

Idea

- Use **hash tables**.
- Replace inner edge lists by `set`s.
- Replace outer list with `dict`.
  - Doesn't speed things up, but allows nodes to have arbitrary labels.

Example




```
{ a: {b},
  b: {c},
  c: {b, d},
  d: {d}
}
```

Time Complexity

operation	code	time
edge query	`j in adj[i]`	$\Theta(1)$ average
degree(i)	`len(adj[i])`	$\Theta(1)$ average

Space Requirements

- Requires only $\Theta(E)$.
- But there is overhead to using hash tables.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 6 | Part 10

Graph Search Strategies

How do we:

- determine if there is a path between two nodes?
- check if graph is connected?
- count connected components?

Search Stategies

- A **search strategy** is a procedure for exploring a graph.
- Different strategies are useful in different situations.

Node Statuses

At any point during a search, a node is in exactly one of three states:

- **visited**
- **pending** (discovered, but not yet visited)
- **undiscovered**

Rules

- At every step, next visited node chosen from among **pending** nodes.
- When a node is marked as **visited**, all of its neighbors have been marked as **pending**.

Choosing the next Node

How to choose among pending nodes?

- Idea 1: Visit **newest** pending (**depth-first search**). DFS
- Idea 2: Visit **oldest** pending (**breadth-first search**). BFS

Main Idea

DFS and BFS each discover different properties of the graph.

For example, we'll see that BFS is useful for finding shortest paths (DFS in general is not).

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 11

Breadth-First Search

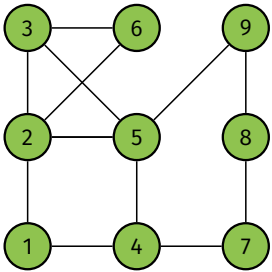
Breadth-First Search

- At every step:
  1. Visit oldest pending node.
  2. Mark its undiscovered neighbors as pending.

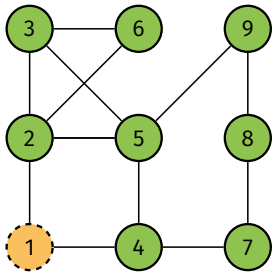
- Convention: in this class, neighbors produced in sorted order.[3]

[3] In general, the order in which a node's neighbors produced is arbitrary.

Example



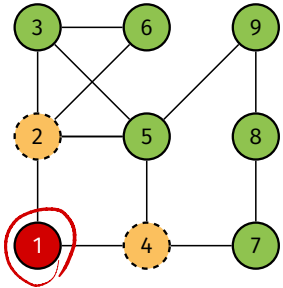
Example




```
pending = [1]
```

Before iterating.

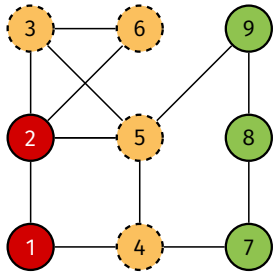
Example




```
pending = [2,4]
```

After 1st iteration.

Example

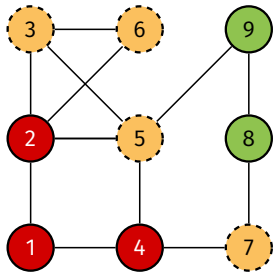



```
pending = [4,3,5,6]
```

After 2nd iteration.

Exercise: what will the picture look like after each of the next two iterations?

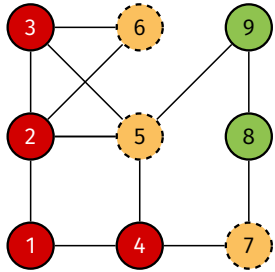
Example




```
pending = [3,5,6,7]
```

After 3rd iteration.

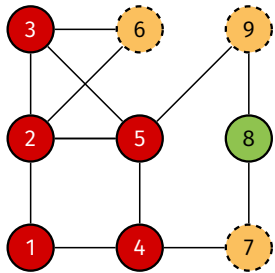
Example




```
pending = [5,6,7]
```

After 4th iteration.

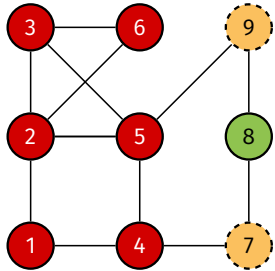
Example




```
pending = [6,7,9]
```

After 5th iteration.

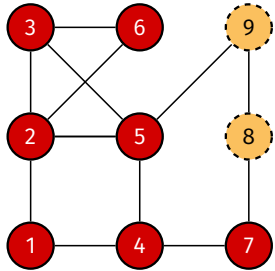
Example




```
pending = [7,9]
```

After 6th iteration.

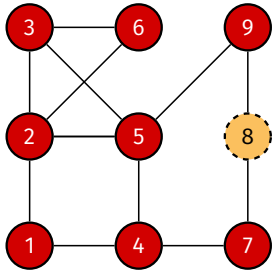
Example




```
pending = [9,8]
```

After 7th iteration.

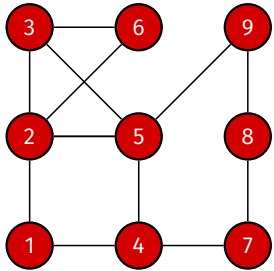
Example




```
pending = [8]
```

After 8th iteration.

Example




```
pending = []
```

After 9th iteration.

Implementation

- To store pending nodes, use a FIFO **queue**.

- While queue is not empty:
  - Pop a node, u.
  - Add undiscovered neighbors to queue.

Queues in Python

- Want $\Theta(1)$ time pops/appends on either side.
- `from collections import deque` (“deck”).
  - `.popleft()` and `.pop()`
  - `list` doesn’t have right time complexity!
  - `import queue` isn’t what you want!
- Keep track of node status attribute using dictionary.

Exercise

```
from collections import deque

def bfs(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}

    status[source] = 'pending'
    pending = deque([source])

    # while there are still pending nodes
    while pending:
        # EXERCISE: fill this in...
```

BFS

```
from collections import deque

def bfs(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}

    status[source] = 'pending'
    pending = deque([source])

    # while there are still pending nodes
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                status[v] = 'pending'
                # append to right
                pending.append(v)
        status[u] = 'visited'
```

Note

- What does this code actually *return*?

Note

- What does this code actually *return*?
- Nothing, yet. It is a *foundation*.

Note

- BFS works just as well for directed graphs.

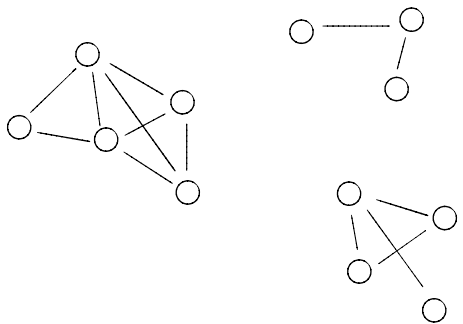
CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 12

Analysis of BFS

Exercise

What will `bfs` do when run on a disconnected graph?



Claim

- `bfs` with source u will visit all nodes reachable from u (and only those nodes).

- Useful!
  - Is there a path between u and v?
  - Is graph connected?

Exploring with BFS

- BFS will visit all nodes reachable from source.
- If **disconnected**, BFS will not visit all nodes.
- We can do so with a **full BFS**.
  - Idea: “re-start” BFS on undiscovered node.
  - Must pass statuses between calls.

Making Full BFS

Modify `bfs` to accept statuses:

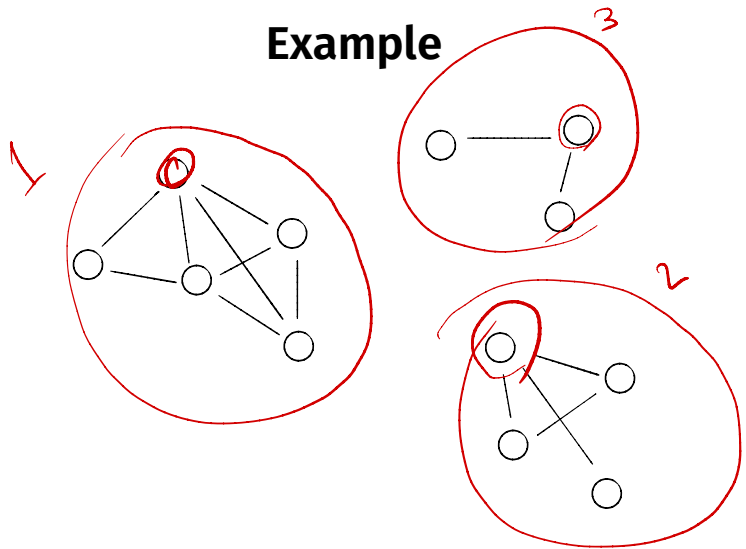
```
def bfs(graph, source, status=None):
    """Start a BFS at `source`."""
    if status is None:
        status = {node: 'undiscovered' for node in graph.nodes}
    # ...
```

Making Full BFS

Call `bfs` multiple times:

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            bfs(graph, node, status)
```

Example



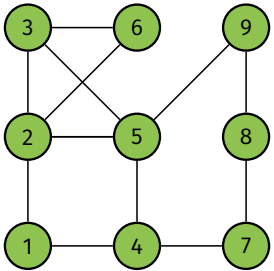
Observation

- If there are k connected components, `bfs` in line 5 is called exactly k times.

```
1  def full_bfs(graph):
2      status = {node: 'undiscovered' for node in graph.nodes}
3      for node in graph.nodes:
4          if status[node] == 'undiscovered':
5              bfs(graph, node, status)
```

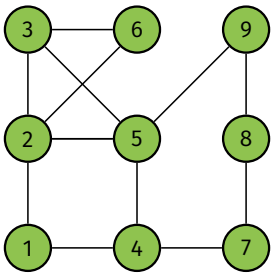
Exercise

How many times is each node added to the queue in a BFS of the graph below? 1



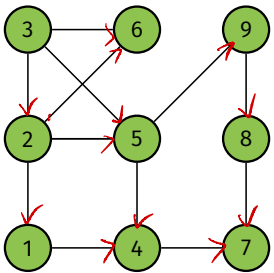
Exercise

How many times is each edge “explored” in a BFS of the graph below? 2



Exercise

How many times is each edge “explored” in a BFS of the *directed* graph below?



Key Properties of `full_bfs`

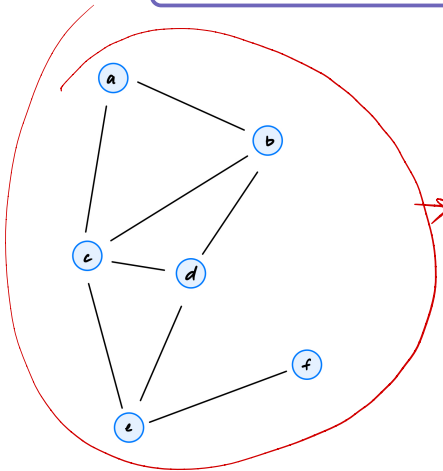
- Each node added to queue **exactly once**.
- Each edge is explored **exactly**:
  - **once** if graph is **directed**.
  - **twice** if graph is **undirected**.

Time Complexity of `full_bfs`

- Analyzing `full_bfs` is easier than analyzing `bfs`.
  - `full_bfs` visits all nodes, no matter the graph.
- Result will be **upper bound** on time complexity of `bfs`.
- We'll use an **aggregate analysis**.

Exercise

What is printed if we run a BFS starting at a?




```
...
while pending:
    u = pending.popleft()
    print(f'Popped {u}')
    for v in graph.neighbors(u):
        print(f'Exploring edge ({u}, {v})')
        # explore edge (u,v)
        ...
```

Answer

$\Theta(|V| + |E|)$

```
Popping a
Exploring edge (a, b)
Exploring edge (a, c)
Popping b
Exploring edge (b, a)
Exploring edge (b, c)
Exploring edge (b, d)
Popping c
Exploring edge (c, a)
Exploring edge (c, b)
Exploring edge (c, d)
Exploring edge (c, e)
```

```
Popping d
Exploring edge (d, b)
Exploring edge (d, c)
Exploring edge (d, e)
Popping e
Exploring edge (e, c)
Exploring edge (e, d)
Exploring edge (e, f)
Popping f
Exploring edge (f, e)
```

Aggregate Analysis

- During any one call to `bfs`:
  - Number of printed nodes: ? $\Theta(|V|)$
  - Number of printed edges: ? $2|E| = \Theta(|E|)$

- In **aggregate** (over all calls):
  - Number of printed nodes: *exactly* $|V|$
  - Number of printed edges: *exactly* $2|E|$

Time Complexity

- Full BFS takes $\Theta(V + E)$

Time Complexity

- Full BFS takes $\Theta(V + E)$
- Why not just $\Theta(E)$?
- $\Theta(V + E)$ works *for all graphs*.
  - If we know more about the number of edges, we might be able to simplify.
  - E.g., if the graph is **complete**, $E = \Theta(V^2)$, so time complexity is $\Theta(V + V^2) = \Theta(V^2)$. If graph is **very sparse**: $\Theta(V)$.

Notational Note

- We'll often write $\Theta(V + E)$ instead of $\Theta(|V| + |E|)$.
- You can use whichever.

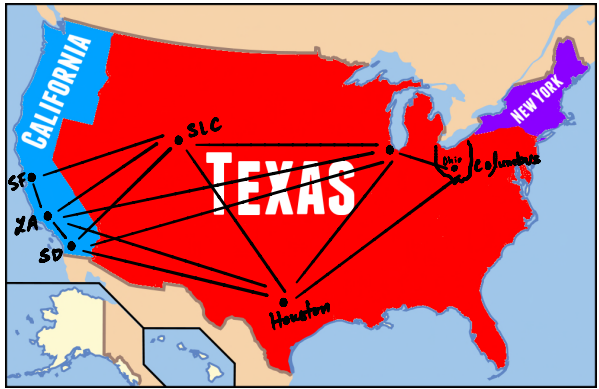
Next

- Finding **shortest paths** using BFS.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 13

Shortest Paths


CALIFORNIA
NEW YORK
TEXAS
SLC
SF
LA
SD
Ohio
Columbus
Houston

Recall

- The **length** of a path is

  (# of nodes) – 1

Definitions

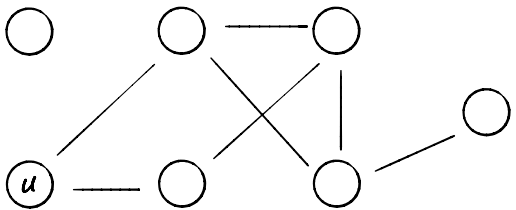
- A **shortest path** between u and v is a path between u and v with smallest possible length.
  - There may be several, or none at all.

- The **shortest path distance** is the length of a shortest path.
  - Convention: ∞ if no path exists.
  - "the distance between u and v" means spd.

Today: Shortest Paths

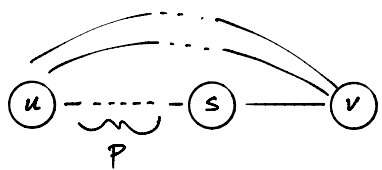
- **Given**: directed/undirected graph G, source u
- **Goal**: find shortest path from u to every other node

Example



Key Property

- A shortest path of length k is composed of:
  - A **shortest path** of length $k - 1$.
  - Plus one edge.



Algorithm Idea

- Find all nodes distance 1 from source.
- Use these to find all nodes distance 2 from source.
- Use these to find all nodes distance 3 from source.
- ...

It turns out...

...this is exactly what BFS does.

CS-GY 6033
Design and Analysis of Algorithms I

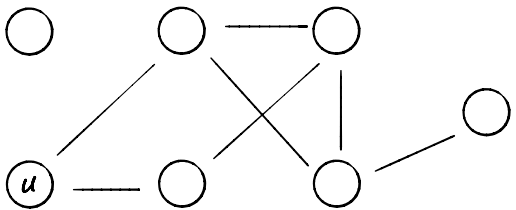
Lecture 6 | Part 14

BFS for Shortest Paths

Key Property of BFS

- For any $k \geq 1$ you choose:
- All nodes distance $k - 1$ from source are added to the queue before any node of distance k.
- Therefore, nodes are “processed” (popped from queue) in order of distance from source.

Example



Discovering Shortest Paths

- We “discover” shortest paths when we pop a node from queue and look at its neighbors.
- But the neighbor’s status matters!

Consider This

- We pop a node *s*.
- It has a neighbor *v* whose status is **undiscovered**.
- We've discovered a **shortest path** to *v* through *s*!

Consider This

- We pop a node *s*.
- It has a neighbor *v* whose status is **pending** or **visited**.
- We already have a shortest path to *v*.

Modifying BFS

- Use BFS “framework”.
- Return dictionary of **search predecessors**.
  - If v is discovered while visiting u, we say that u is the **BFS predecessor** of v.
  - This encodes the shortest paths.
- Also return dictionary of shortest path distances.

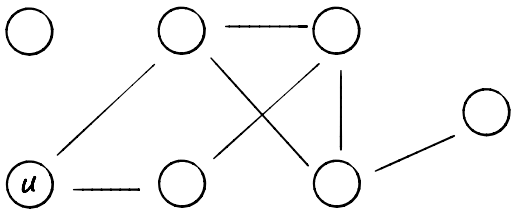
```python
def bfs_shortest_paths(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
    distance = {node: float('inf') for node in graph.nodes}
    predecessor = {node: None for node in graph.nodes}

    status[source] = 'pending'
    distance[source] = 0
    pending = deque([source])

    # while there are still pending nodes
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                status[v] = 'pending'
                distance[v] = distance[u] + 1
                predecessor[v] = u
                # append to right
                pending.append(v)
        status[u] = 'visited'

    return predecessor, distance
```

Example



CS-GY 6033
Design and Analysis of Algorithms I

Lecture 6 | Part 15

BFS Trees

Result of BFS

- Each node reachable from source has a single BFS predecessor.
  - Except for the source itself.

- The result is a **tree** (or forest).

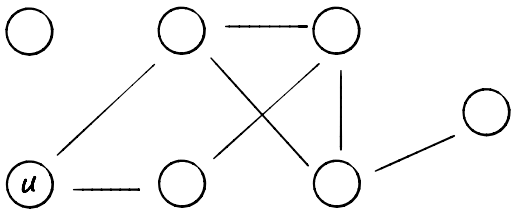
Trees

- A (free) **tree** is an undirected graph $T = (V, E)$ such that T is connected and $|E| = |V| - 1$.
- A **forest** is graph in which each connected component is a tree.

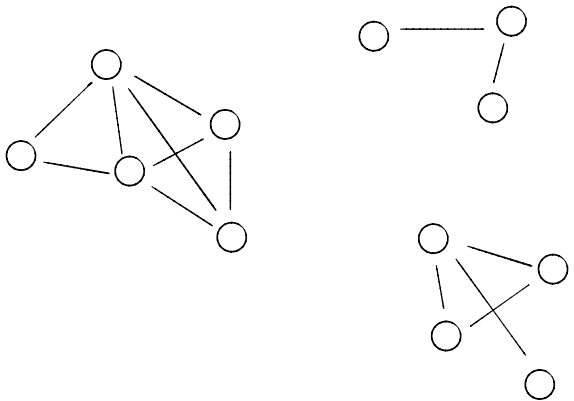
BFS Trees (Forests)

- If the input is connected, BFS produces a **tree**.
- If the input is not connected, BFS produces a **forest**.

Example



Example



BFS Trees

- BFS trees and forests encode shortest path distances.